SIMATS ENGINEERING

ASSESSMENT TOOL 1 -CONSTRAINT BASED PROBLEM SOLVING

COURSE NAME: R PROGRAMMING **COURSE CODE: ITA0401**

COURSE OUTCOME COVERED

CO1: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively. Identify and implement appropriate control structures to solve a particular programming problem

(Bloom Level -BL3- BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Questions **Total Marks: 100**

Code of Conduct:

I T. Sai Praneeth · 192525434 certify that this submission is my original work and that I have adhered to the *guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

T. Sai Praneeth

Signature of the student:

2. EMPLOYEE SALARY PROCESSING

TASK AND LEARNING OUTCOMES

Represent employee ID, name, basic pay, and allowance; calculate gross salary; list employees earning above ₹50,000; report the highest-paid employee; and generate a summary. A data frame is appropriate for rectangular records with mixed types.

ASSUMPTIONS AND ALGORITHM

Gross salary is Basic Pay + Allowance; taxes and deductions are outside scope. The condition is strictly > ₹50,000. The maximum selection includes all employees tied at the maximum. The data is synthetic and uses whole-rupee amounts.

R SOLUTION

```
employees <- data.frame(
  employee_id=c("E101","E102","E103","E104","E105"),
  name=c("Anita","Bharat","Charu","Deepak","Farah"),
  basic_pay=c(42000,48000,36500,52000,44000),
  allowance=c(8500,6000,7000,9000,5500),
  stringsAsFactors=FALSE)
employees$gross_salary <- employees$basic_pay + employees$allowance
cat("PAYROLL\n"); print(employees, row.names=FALSE)
cat("ABOVE INR 50,000\n")
print(subset(employees, gross_salary > 50000), row.names=FALSE)
maximum_salary <- max(employees$gross_salary)
highest_paid <- subset(employees, gross_salary == maximum_salary)
cat("HIGHEST PAID\n"); print(highest_paid, row.names=FALSE)
summary_report <- data.frame(
  headcount=nrow(employees), mean_gross=mean(employees$gross_salary),
  median_gross=median(employees$gross_salary),
  minimum_gross=min(employees$gross_salary), maximum_gross=maximum_salary)
print(summary_report, row.names=FALSE)
```

EXPECTED CHECKS AND INTERPRETATION

Gross salaries are ₹50,500, ₹54,000, ₹43,500, ₹61,000, and ₹49,500. Four employees exceed ₹50,000; Deepak has the highest salary at ₹61,000. Summary measures should be checked against these five values.

ASSESSMENT AND PRACTICAL NOTE

Award 2 marks for a suitable structure, 2 for salary calculation, 2 for filtering, 2 for maximum/ties, and 2 for a meaningful summary and explanation. In a real payroll workflow, define handling for missing or negative values, deductions, privacy, and currency formatting.

3. WEATHER DATA ANALYSIS USING ARRAYS

TASK AND LEARNING OUTCOMES

Store seven daily temperature readings for three cities in a three-dimensional array. Find city averages, identify the hottest day, and compare city-wise temperature variation. The dimensions are City × Day × Measure, with one temperature measure.

```
print(inventory, row.names=FALSE)
print(inventory[inventory$low_stock, ], row.names=FALSE)
summary <- data.frame(
 products=nrow(inventory), units=sum(inventory$quantity),
 inventory_value=sum(inventory$value), low_stock=sum(inventory$low_stock))
print(summary, row.names=FALSE)
```

EXPECTED CHECKS AND ASSESSMENT

Low-stock items are Pen Pack (8) and Calculator (5). Total quantity is 63; total inventory value is ₹7,715; two products are below threshold. Ruler has exactly 10 and is not flagged. Marks: list/vector design 2; line values 2; threshold filter 2; summary 2; aligned fields, labels, explanation 2.

For maximum compatibility, the length check is written without newer pipe syntax. In real stock control, also validate nonnegative quantities, price units, and missing values.

5. BANK ACCOUNT PROCESSING USING FUNCTIONS

TASK AND LEARNING OUTCOMES

Define functions to deposit, withdraw, check balance, and display transaction history. The account is an in-memory list containing a balance and a transaction data frame. Each successful operation returns the updated account, which the caller must assign.

RULES AND VALIDATION

Transaction amounts must be one finite positive number. An overdraft is rejected and must not be recorded. This makes validation observable and prevents a failed transaction from silently altering account state.

R SOLUTION

```
new_account <- function(opening=0) {
 stopifnot(length(opening)==1, is.finite(opening), opening>=0)
 list(balance=as.numeric(opening), history=data.frame(
   type=character(), amount=numeric(), balance_after=numeric()))
}
record_tx <- function(a, type, amount, new_balance) {
 a$balance <- new_balance
 a$history <- rbind(a$history, data.frame(type=type, amount=amount,
                                          balance_after=new_balance))
 a
}
deposit <- function(a, amount) {
 if (length(amount)!=1 || !is.finite(amount) || amount<=0)
   stop("Deposit must be positive")
 record_tx(a,"DEPOSIT",amount,a$balance+amount)
}
withdraw <- function(a, amount) {
 if (length(amount)!=1 || !is.finite(amount) || amount<=0)
   stop("Withdrawal must be positive")
 if (amount>a$balance) stop("Insufficient funds; no transaction recorded")
 record_tx(a,"WITHDRAWAL",amount,a$balance-amount)
}
check_balance <- function(a) a$balance
display_history <- function(a) print(a$history, row.names=FALSE)
```

R SOLUTION

```
citizens <- data.frame(
 name=c("Aarav","Diya","Ishaan","Meera","Kabir","Nila","Rohan","Sara"),
 age=c(17,18,42,16,21,18,15,65), stringsAsFactors=FALSE)
citizens$eligible <- citizens$age >= 18
n_eligible <- sum(citizens$eligible)
n_ineligible <- sum(!citizens$eligible)
pct <- 100 * n_eligible / nrow(citizens)
print(citizens, row.names=FALSE)
cat("Eligible:",n_eligible," Ineligible:",n_ineligible,
    " Percent:",sprintf("%.1f%%",pct),"\n")
```

EXPECTED RESULT, EDGE CASES, MARKING

Five of eight are eligible, three are ineligible, and the percentage is 62.5%. Exactly age 18 qualifies because >= is used. If age can be missing, the resulting NA should be reported as unknown or handled by an explicit policy, not silently counted. Marks: data set 2; logical vector 2; counts 2; percentage 2; boundary and missing-age explanation 2.

This demonstrates only the age rule given in the assignment. Actual voter eligibility depends on the applicable jurisdiction and other legal requirements.

8. MATRIX-BASED SALES ANALYSIS

TASK AND LEARNING OUTCOMES

Represent sales for four products across three regions in a matrix; compute product and region totals; find the strongest product and region. Rows are products and columns are regions, so rowSums() produces product totals and colSums() produces regional totals.

INTERPRETATION

The prompt says monthly sales but gives only a product × region matrix. This solution treats it as one month. To compare multiple months, add a Month dimension in an array or retain a long-format data frame. No currency or unit is assumed.

R SOLUTION

```
sales <- matrix(c(
 120,150,130,
 200,180,220,
 160,140,170,
  90,110,100), nrow=4, byrow=TRUE,
 dimnames=list(Product=c("Product_A","Product_B","Product_C","Product_D"),
               Region=c("North","South","West")))
print(sales)
product_total <- rowSums(sales)
region_total <- colSums(sales)
print(product_total); print(region_total)
cat("Best product:",names(which.max(product_total)),
    max(product_total),"\n")
cat("Top region:",names(which.max(region_total)),
    max(region_total),"\n")
stopifnot(sum(product_total)==sum(region_total))
```

10. UNIVERSITY INFORMATION SYSTEM USING S4 AND REFERENCE CLASS

TASK AND LEARNING OUTCOMES

Define an S4 Student with roll number, name, and CGPA slots, then a reference-class registry to add records, modify CGPA, display students, and find CGPA > 8.0. Roll numbers are unique; CGPA must lie in [0,10]. The comparison is strict, so exactly 8.0 does not qualify.

R SOLUTION: S4 MODEL AND REFERENCE REGISTRY

```
setClass("Student",
 slots=c(roll_number="character",name="character",cgpa="numeric"),
 validity=function(object) {
  if(length(object@roll_number)!=1 || !nzchar(object@roll_number))
   return("roll_number must be non-empty")
  if(length(object@name)!=1 || !nzchar(object@name)) return("name must be non-empty")
  if(length(object@cgpa)!=1 || !is.finite(object@cgpa) ||
     object@cgpa<0 || object@cgpa>10) return("CGPA must be from 0 to 10")
  TRUE
 })
Registry <- setRefClass("Registry", fields=list(records="list"), methods=list(
 add_student=function(roll,name,cgpa) {
  if(any(vapply(records,function(s)s@roll_number==roll,logical(1))))
   stop("Duplicate roll number")
  records[[length(records)+1L]] <<- new("Student",roll_number=roll,
    name=name,cgpa=as.numeric(cgpa)); invisible(.self)
 },
 modify_cgpa=function(roll,value) {
  i <- which(vapply(records,function(s)s@roll_number==roll,logical(1)))
  if(length(i)!=1) stop("Student not found")
  s <- records[[i]]; s@cgpa <- as.numeric(value); validObject(s)
  records[[i]] <<- s; invisible(.self)
 },
 display=function() {
  for(s in records) cat(s@roll_number,"|",s@name,"|",s@cgpa,"\n")
 },
 high_achievers=function(threshold=8) Filter(function(s)s@cgpa>threshold,records)
))
```

R SOLUTION: SAMPLE EXECUTION

```
r <- Registry$new(records=list())
r$add_student("U001","Asha",8.4)
r$add_student("U002","Dev",7.8)
r$add_student("U003","Ira",8.0)
r$modify_cgpa("U002",8.2)
r$display()
for(s in r$high_achievers(8.0))
 cat("Above 8:",s@roll_number,s@name,s@cgpa,"\n")
```

EXPECTED CHECKS AND ASSESSMENT

Asha (8.4) and Dev (8.2) qualify; Ira (8.0) does not. Duplicate IDs and out-of-range CGPA values must fail. validObject() checks an edited instance before the registry is updated. Marks: valid S4 design 2; reference registry 2; add/display 2; update/lookup 2; strict threshold and tests 2.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand